

Dog Explorer

רקע

במשימה זו תבנה אפליקציית Frontend להצגת גזעי כלבים ותמונות כלבים באמצעות Dog API.

האפליקציה תאפשר למשתמש:

- לקבל תמונת כלב אקראית
- לצפות ברשימת גזעים
- לחפש גזע
- להיכנס לעמוד של גזע מסוים
- לצפות בגלריית תמונות של אותו גזע
- להוסיף תמונות למועדפים
- לצפות במועדפים גם לאחר רענון הדף

המשימה היא משימת Frontend בלבד. אין צורך לבנות שרת או מסד נתונים.

מטרות המשימה

המטרה היא לתרגל עבודה מעשית עם:

- Vue 3
- Vue Router
- Pinia
- עבודה מול API חיצוני
- חלוקה לקומפוננטות
- Props ו-Emits
- State מקומי ו-State גלובלי
- localStorage
- טיפול בטעינה, שגיאות ומצבים ריקים
- ארגון פרויקט בצורה מסודרת

טכנולוגיות

יש להשתמש ב:

- Vue 3
- Vite
- Vue Router

Pinia -
JavaScript -
CSS -
Axios -

אין להשתמש ב-TypeScript.

ה-API

יש להשתמש ב-Dog API:

<https://dog.ceo/dog-api>

ה-API מחזיר רשימות גזעים ותמונות כלבים לפי גזע.

Endpoints מרכזיים:

קבלת תמונת כלב אקראית

GET <https://dog.ceo/api/breeds/image/random>

קבלת מספר תמונות אקראיות

GET <https://dog.ceo/api/breeds/image/random/6>

ניתן לבקש עד 50 תמונות בבקשה אחת.

קבלת רשימת כל הגזעים

GET <https://dog.ceo/api/breeds/list/all>

קבלת כל התמונות של גזע

GET <https://dog.ceo/api/breed/{breed}/images>

לדוגמה:

GET <https://dog.ceo/api/breed/hound/images>

ה-API תומך גם בגזעים ובתתי-גזעים.

מבנה האפליקציה

האפליקציה תכלול:

Header שבו מופיע

- קישורים לעמודי הבית, הזנים והמועדפים

- שם העמוד הנוכחי

- הסבר קצר על העמוד

בקישור לעמוד המועדפים יש להציג את מספר המועדפים:

יש להשתמש ב-"RouterLink" ולא בקישורי HTML רגילים.

האפליקציה תכלול את העמודים הבאים:

```
/
/breeds
/breeds/:breed
/favorites
/:pathMatch(.*)*
```

עמוד הבית

נתיב:

```
/
```

דרישות

עמוד הבית יציג:

- תמונת כלב אקראית

- שם הגזע של הכלב בתמונה האקראית

- כפתור לקבלת תמונה חדשה

התנהגות

בעת כניסה לעמוד:

1. יש לבצע בקשה לקבלת תמונה אקראית.

2. בזמן הבקשה יש להציג מצב טעינה.

3. לאחר קבלת התמונה יש להציג אותה.

4. אם הבקשה נכשלה, יש להציג הודעת שגיאה וכפתור ניסיון חוזר.

5. בזמן טעינת תמונה חדשה, הכפתור צריך להיות disabled.

עמוד רשימת הגזעים

נתיב:

/breeds

דרישות

העמוד יציג:

- שדה חיפוש
- רשימת כל הגזעים
- מספר הגזעים שנמצאו
- קישור לעמוד של כל גזע

חיפוש

החיפוש יהיה לפי שם הגזע.

לדוגמה:

hus

יציג:

husky

החיפוש צריך:

- להתעלם מאותיות גדולות וקטנות
- להתעדכן בזמן ההקלדה
- להתבצע על הנתונים שכבר התקבלו מה-API
- לא לבצע קריאת API בכל הקלדה

הרשימה המסוננת צריכה להיות מחושבת באמצעות computed.

כאשר אין גזעים מתאימים, יש להציג הודעה ברורה.

תתי־גזעים

תשובת ה-API מחזירה אובייקט שבו:

- המפתח הוא שם הגזע
- הערך הוא מערך תתי־גזעים

אין צורך לבנות עמוד נפרד לתת־גזע, או להציג אותם בעמוד זה.

עמוד פרטי גזע

נתיב:

/breeds/:breed

לדוגמה:

/breeds/husky

דרישות

העמוד יציג:

- שם הגזע מתוך ה-URL
- גלריית תמונות של אותו גזע

טעינת התמונות

יש לקבל את שם הגזע דרך Vue Router:

route.params.breed

לאחר מכן יש לשלוח בקשה מתאימה ל-API.

אין צורך להציג את כל התמונות שה-API מחזיר.

יש לבחור אחת מהאפשרויות:

- להציג עד 20 תמונות
- להציג מספר תמונות אקראיות מהגזע
- להוסיף כפתור "טען תמונות נוספות"

גזע לא קיים

אם המשתמש נכנס לכתובת של גזע שלא קיים:

breeds/not-a-real-breed/

יש להציג הודעה ברורה ולא להשאיר מסך ריק.

עמוד מועדפים

נתיב:

/favorites

דרישות

העמוד יציג

- כל התמונות שהמשתמש הוסיף למועדפים
- מספר המועדפים

לכל מועדף יש להציג:

- תמונת הכלב
- שם הגזע
- קישור לעמוד הגזע

מצב ריק

כאשר אין מועדפים, יש להציג Empty State.

לדוגמה:

עדיין לא שמרת תמונות למועדפים.

ניקוי מועדפים

יש להוסיף כפתור:

נקה את כל המועדפים

לפני הניקוי יש לבקש אישור מהמשתמש, מומלץ להשתמש ב Dialog.

עמוד לא נמצא

נתיב

/pathMatch:()

יש ליצור עמוד "NotFound".

העמוד יציג:

- הודעת 404
- הסבר קצר
- קישור לעמוד הבית

שמירת המועדפים

המועדפים ינוהלו באמצעות Pinia ויישמרו ב localStorage.

דרישות

יש לממש:

- הוספת מועדף
- הסרת מועדף
- בדיקה האם תמונה כבר נמצאת במועדפים
- ניקוי כל המועדפים
- מספר המועדפים
- טעינת המועדפים

מניעת כפילויות

אותה תמונה לא יכולה להופיע פעמיים במועדפים.

Pinia Store

State מקומי מול גלובלי

יש להשתמש ב-Pinia רק עבור מידע שמשותף בין מספר עמודים או קומפוננטות.

לדוגמה:

מתאים ל-Pinia:

- רשימת המועדפים
- רשימת הגזעים
- מספר המועדפים

לא חייב להיות ב-Pinia:

- הטקסט הנוכחי בשדה החיפוש
- האם Dialog מקומי פתוח
- מספר התמונות המוצגות בעמוד מסוים
- מצב hover של כרטיס

Router View

העמודים יוצגו דרך:

</ RouterView>

נתיב לא קיים

יש ליצור עמוד "NotFound".

העמוד יציג:

- הודעת 404

- הסבר קצר

- קישור לעמוד הבית

שכבת Service

יש ליצור קובץ נפרד שאחראי על העבודה מול ה-API.

דרישות

- כתובת הבסיס של ה-API תוגדר במקום אחד
- אין לפזר כתובות API בתוך קומפוננטות שונות
- הפונקציות יחזירו מידע או יזרקו שגיאה
- אין להציג הודעות UI מתוך קובץ ה-service

קומפוננטות

יש לפרק את האפליקציה לקומפוננטות חוזרות.

קומפוננטות אפשריות

/components

AppHeader.vue

DogImageCard.vue

BreedCard.vue

SearchInput.vue

LoadingState.vue

ErrorState.vue

EmptyState.vue

אין חובה להשתמש בדיוק בשמות האלה.

DogImageCard

כל תמונת כלב באתר תכלול כפתור **toggle favorite**

פירוק נכון

אין צורך להפוך כל כפתור לקומפוננטה.

יש ליצור קומפוננטה כאשר:

- נעשה בה שימוש ביותר ממקום אחד
- יש לה אחריות ברורה
- היא מורכבת מספיק כדי להצדיק קובץ נפרד

מצבי מסך

כל עמוד שמבצע קריאה ל-API חייב לטפל בארבעה מצבים:

Loading

הבקשה עדיין מתבצעת.

לדוגמה:

טוען כלבים...

Success

המידע התקבל בהצלחה.

Error

הבקשה נכשלה.

יש להציג:

- הודעת שגיאה ידידותית
- כפתור ניסיון חוזר, כאשר אפשר

אין להסתפק ב:

console.error

console.error יכול להישאר לצורכי פיתוח, אך המשתמש צריך לקבל משוב במסך.

Empty

הבקשה הצליחה אך אין מידע להצגה.

עיצוב וחווית משתמש UI/UX

אין צורך לבנות עיצוב מורכב, אך האפליקציה צריכה להיות ברורה ונוחה לשימוש.

דרישות בסיסיות

- פריסה מסודרת
- התאמה **בסיסית** למובייל
- תמונות בגודל אחיד
- תמונות לא יעוותו
- כפתורים ברורים
- מצב disabled בזמן פעולה
- סימון ברור של תמונה מועדפת
- Header קבוע
- מרווחים אחידים
- Empty State ברור
- Error State ברור

תמונות

מומלץ להשתמש ב:

;object-fit: cover

כדי לשמור על כרטיסים בגודל אחיד.

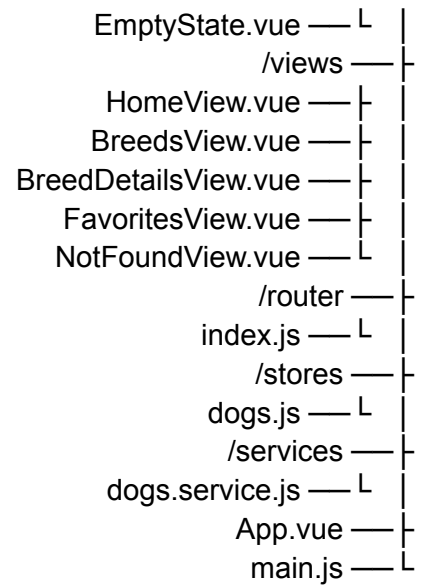
נגישות בסיסית

- לכל תמונה יהיה "alt"
- כפתורים יכילו טקסט וכמו כן תיאור ב hover.
- אלמנט לחיצה יהיה "a", "button", או "RouterLink"
- לא להשתמש ב-"div" כלחצן בלי סיבה
- ניתן להשתמש באפליקציה גם עם מקלדת ברמה בסיסית

מבנה תיקיות מומלץ

```

/src
  /assets
  /components
    AppHeader.vue
    BreedCard.vue
    DogImageCard.vue
    LoadingState.vue
    ErrorState.vue
```



ניתן לבחור מבנה אחר, כל עוד הוא עקבי וברור.

שלבי ביצוע

יש לנהל את הפרויקט באמצעות github.
כל שלב צריך branch נפרד.
בסוף כל שלב יש לקבל merge approval כדי לעבור לשלב הבא.

שלב 1 — הקמת הפרויקט

יש לבצע:

- יצירת פרויקט Vue
- התקנת Vue Router
- התקנת Pinia
- יצירת מבנה תיקיות
- יצירת Header
- יצירת הנתיבים הראשוניים
- יצירת עמוד 404

Break Point 1

יש להציג:

- אפליקציה שעולה בהצלחה
- מעבר בין כל העמודים
- Header משותף
- Route דינמי
- עמוד 404

ה-API עדיין לא חייב להיות מחובר.

שלב 2 — תמונה אקראית

יש לבצע:

- יצירת שכבת Service
- קריאה לתמונה אקראית
- הצגת התמונה בעמוד הבית
- כפתור לקבלת תמונה חדשה
- Loading
- Error
- Retry

Break Point 2

יש להציג Flow מלא:

1. כניסה לעמוד הבית
2. הצגת Loading
3. הצגת תמונה
4. החלפת תמונה
5. הדגמת שגיאה וטיפול בה

שלב 3 — רשימת גזעים

יש לבצע:

- קבלת רשימת גזעים
- שמירת הרשימה
- הצגת הרשימה
- חיפוש
- "computed" לסינון
- מצב ללא תוצאות
- ניווט לעמוד גזע

Break Point 3

יש להציג:

- רשימת גזעים
- חיפוש תקין
- ניווט באמצעות Router

שלב 4 — עמוד גזע

יש לבצע:

- קריאת "breed" מה-route
- שליפת תמונות לפי גזע
- גלריה
- Loading
- Error
- גזע לא קיים

Break Point 4

יש להציג:

- כניסה דרך רשימת הגזעים
- כניסה ישירה דרך כתובת URL
- טיפול בגזע שלא קיים
- רענון הדף בעמוד גזע

שלב 5 — Pinia ומועדפים

יש לבצע:

- יצירת Store
- הוספת תמונה למועדפים
- הסרת תמונה
- מניעת כפילויות
- מספר מועדפים ב-Header
- עמוד מועדפים
- Empty State

Break Point 5

יש להציג:

1. הוספת תמונה בעמוד הבית
2. מעבר לעמוד גזע והוספת תמונה נוספת
3. הצגת שתי התמונות בעמוד המועדפים
4. הסרת תמונה
5. עדכון המספר ב-Header

שלב 6 — localStorage

יש לבצע:

- שמירת המועדפים
- טעינת המועדפים בעליית האפליקציה
- שמירה לאחר כל שינוי
- טיפול במידע לא תקין
- ניקוי כל המועדפים

Break Point 6

יש להציג:

1. הוספת מועדפים
2. רענון הדף
3. המועדפים עדיין קיימים
4. סגירה ופתיחה מחדש של האפליקציה
5. המועדפים עדיין קיימים
6. טיפול בערך "localStorage" לא תקין

שלב 7 — ניקוי וסיום

יש לבצע:

- ניקוי קוד
- הוצאת קוד חוזר
- בדיקת שמות
- בדיקת Console
- Responsive בסיסי
- README
- בדיקה ידנית של כל ה-Flows

Break Point סופי

יש להציג את האפליקציה מתחילתה ועד סופה.

קריטריוני קבלה

המשימה תיחשב מושלמת כאשר:

Vue

- האפליקציה בנויה באמצעות Vue 3
- קיימת חלוקה הגיונית לקומפוננטות
- יש שימוש ב-Props
- יש שימוש ב-Emits
- יש שימוש ב-"computed"

- אין שכפול משמעותי של קוד

Router

- כל הנתיבים עובדים
- קישורי הניווט משתמשים ב-"RouterLink"
- עמוד גזע משתמש ב-Route Param
- כניסה ישירה ל-URL עובדת
- קיים עמוד 404

Pinia

- המועדפים מנוהלים ב-Pinia
- מספר המועדפים מחושב דרך Store
- קיימת מניעת כפילויות
- ניתן להוסיף ולהסיר מועדפים
- State מקומי אינו מוכנס ל-Pinia ללא צורך

API

- הקריאות נמצאות בשכבת Service
- קיימים Loading ו-Error
- קיימת אפשרות Retry
- כתובת ה-API אינה מפוזרת בפרויקט
- אין קריאות API מיותרות

localStorage

- המועדפים נשמרים
- המועדפים נטענים לאחר רענון
- מידע לא תקין אינו מפיל את האפליקציה
- ניקוי המועדפים מעדכן גם את Pinia וגם את "localStorage"

UX

- קיימים מצבי Loading
- קיימים מצבי Error
- קיימים Empty States
- האפליקציה עובדת במובייל בצורה בסיסית
- התמונות מוצגות בצורה אחידה
- אין שגיאות Console משמעותיות

README

יש להוסיף קובץ "README.md" שיקלול:

- שם הפרויקט
- תיאור קצר
- הטכנולוגיות שבהן השתמשת
- הוראות התקנה
- הוראות הפעלה
- מבנה העמודים
- תיאור קצר של ה-Store
- תיאור העבודה מול "localStorage"
- קישור ל-Dog API
- צילום מסך אחד לפחות

הוראות הפעלה לדוגמה

```
npm install  
npm run dev
```

מגבלות

אין צורך להוסיף:

- Backend
- מסד נתונים
- Authentication
- TypeScript
- Tests אוטומטיים
- SSR
- Nuxt
- WebSocket
- מערכת משתמשים
- העלאת תמונות
- מערכת הרשאות

המטרה היא לבצע היטב את הנושאים שכבר נלמדו, ולא להוסיף כמה שיותר טכנולוגיות.